L11: Deep Learning for Text

CSci 560 Deep Learning w/ Python (Chollet) Ch. 11

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L11.1 Natural Language Processing: The bird's eye view

CSci 560: Deep Learning w/ Python (Chollet) Ch. 11.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Natural vs. Computer Languages

- In computer science, we refer to human languages, like English or Mandarin, as **natural** languages, to distinguish them from languages designed for machines.
- Every machine language was **designed**: its starting point was a human engineer writing down a se of formal rules to describe the language.
  - Rules came first.
- Natural languages are shaped by an evolutionary process, much like biological organisms, that's whats make them *natural*.
  - Their "rules", like the grammar of English, were formalized after the fact and are often ignored or broken by its users.

Conclusion: While machine-readable languages are highly structured and rigorous, natural language is messy - ambiguous, chaotic, sprawling, and constantly in flux.

EAST TEXAS A&M

Modern Machine Learning NLP Systems

Using machine learning and large datasets to give computers the ability to *understand* language.

Some examples of types of text processing systems:

- "What's the topic of this text?" (text classification)
- "Does this text contain abuse?" (content filtering)
- "Does this text sound positive or negative?" (sentiment analysis)
- "What should be the next word in this incomplete sentence?" (language modeling)
- "How would you say this in German?" (translation)
- "How would you summarize this article in one paragraph?" (summarization)

In much the same way that computer vision is pattern recognition applied to pixels, NLP is pattern recognition applied to words, sentences, and paragraphs.

EAST TEXAS A&M

L11.2 Preparing Text Data

CSci 560: Deep Learning w/ Python (Chollet) Ch. 11.2

Derek Harter

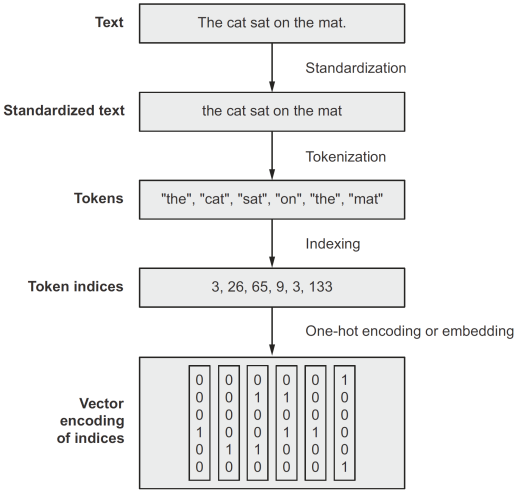
Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Text Vectorization Process

Vectorizing text is the process of transforming text into numeric tensors.



- First, you standardize the text to make it easier to process, such as by converting it to lowercase or removing punctuation.
- You split the text into units (called tokens), such as characters, words, or groups of words. This is called tokenization.
- You convert each such token into a numerical vector. This will usually involve first indexing all tokens present in the data.

EAST TEXAS A&M

Text Standardization

- A machine learning model doesn't know a priori that "i" and "I" are the same letter, that "é" is an "e" with an accent, or that "staring" and "stared" are two forms of the same verb.
- Text standardization is a basic form of feature engineering that aims to erase encoding differences that you don't want your model to have to deal with.
- Simplest and usually used: convert to lowercase and remove punctuation characters.
- Also special characters may be standard form, such as replacing accented vowels with unaccented form.
- **stemming** converting variations of a teerm into a single shared representation
  - turn "caught" and "been catching" into "[catch]"
  - plurals often stemmed, "cats" into "[cat]"
- With these standardization techniques, your model will require less training data and will generalize better.

EAST TEXAS A&M

Text splitting (tokenization)

Tokenization: break up text into units to be vectorized.

Three different approaches:

1. **Word-level tokenization** Where tokens are space-separated (or punctuation separated) substrings. A variant of this is to further split words into subwords when applicable for instance, treating "staring" as "star+ing" or "called" as "call+ed."
2. **N-gram tokenization** Where tokens are groups of N consecutive words. For instance, "the cat" or "he was" would be 2-gram tokens (also called bigrams).
3. **Character-level tokenization** Where each character is its own token. In practice, this scheme is rarely used, and you only really see it in specialized contexts, like text generation or speech recognition.

EAST TEXAS A&M

Text-processing Model Types: sequence vs. bag-of-words

There are two kinds of text-processing models:

1. Those that care about word order are called **sequence models**.
2. Those that treat input words as a set, discarding original order, are called **bag-of-words models**.

If building a sequence model, you'll use word-level tokenization.

If building a bag-of-words model, you'll use N-gram tokenization.

EAST TEXAS A&M

Understanding N-grams and bag-of-words

- Word N-grams are groups of N (or fewer) consecutive words.
- For example, given sentence "the cat sat on the mat."
- 2-grams (or bigrams): `['the cat', 'cat sat', 'sat on', 'on the', 'the mat']`
- 3-grams (or trigrams): `['the cat sat', 'cat sat on', 'sat on the', 'on the mat']`

Because bag-of-words isn't an order-preserving tokenization method (the tokens generated are understood as a set, not a sequence, and the general structure of the sentences is lost), it tends to be used in shallow language-processing models rather than in deep learning models.

EAST TEXAS A&M

Vocabulary Indexing

- Once your text is split into tokens, you need to encode each token into a numerical representation.
- You could potentially do this in a stateless way, such as by hashing each token into a fixed binary vector
- In practice, the way you'd go about it is to build an index of all terms found in the training data
- It's common to restrict the vocabulary to the top N most frequent words, for example 10,000
- When we look up a new token in the vocabulary index, it may ot necessarily exist.
  - To handle this, you should use an "out of vocabulary" index (abbreviated as OOV index), a catch-all for any token that wasn't in the index.
  - **OOV token**: "out of vocabulary", usually token index 1
  - **mask token**: "ignore me not a word", usually token index 0

EAST TEXAS A&M

L11.3 Two Approaches for Representing Groups of Words: sets and sequences

CSci 560: Deep Learning w/ Python (Chollet) Ch. 11.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

How to Encode Word Order

- How to represent **individual words** is relatively uncontroversial: they're categorical features and we assign an integer index (dimension number) and vectorize (encode as dimensions in a feature space).
- More problematic question: how to encode **the way words are woven into sentences**: word order.
- Unlike many other types of timeseries, words in most natuaral languages don't have a canonical order.
  - Order is clearly important, but its relationship to meaning (in natural languages) isn't straight forward.

- Simplest approach: discard order and treat as unordered set: **bag-of-words models**
- Process strictly in order they appear, like steps in a timeseries: **sequence models**
- Hybrid approach: **Transformer architecture** technically order-agnostic, yet injects word-postiion info into representations.

EAST TEXAS A&M

Understanding Word Embeddings

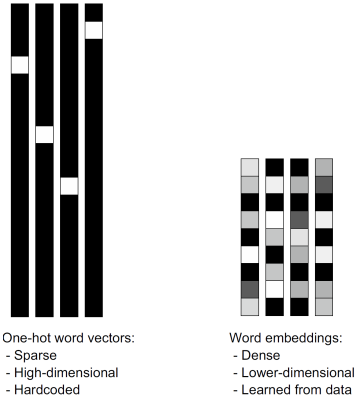


Figure 2: Word representations obtained from one-hot encoding are sparse, high-dimensional and hardcoded. Word embeddings are dense, relatively low-dimensional, and learned from data. (Chollet, 2021, pg 330)

- One-hot encoding assumes that different okens are **all independent from each other**. (one-hot vectors are all orthonal to one another).
- The **geometric relationship** between two word vectors should reflect the **semantic relationship** between these words.
  - In a reasonable word vector space, similar words should be encoded in similar vectors.
  - e.g. the vector for "film" and "movie" should be about the same vector.
- **Word embeddings** are vector representations of words that achieve exactly this: they map human language into a structured geometric space.

EAST TEXAS A&M

Example of Word Embedding Space

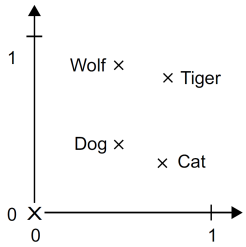


Figure 3: A toy example of a word embedding space.(Chollet, 2021, pg.330)

- 4 words embedded on a 2D plane: cat, dog, wolf and tiger
- Same vector from "dog" to "wolf" and "cat" to "tiger"
  - semantics "from pet to wild animal"
- Similar same vector from "dog" to "cat" as "wolf" to "tiger"
  - semantics "from canine to feline"

EAST TEXAS A&M

Two Ways to Obtain Word Embeddings

- Learn word embeddings jointly with the main task you care about (such as document classification or sentiment prediction).
  - In this setup, you start with random word vectors and then learn word vectors in the same way you learn the weights of a neural network.
- Load into your model word embeddings that were precomputed using a different machine learning task than the one you're trying to solve.
  - These are called pretrained word embeddings.

EAST TEXAS A&M

Summary of Representing Groups of Words

- Word order in Text processing is handled in 2 basic ways:
  1. **bag-of-words models** : discard order and treat as an unordered set (multi-hot encoding typically, 20,000 sparse vectors).
  2. **sequence models**: process words in order they appear like a timeseries
- For sequence models, can encode sequence again using one-hot encoding, but this ends up with very large input to RNN models.
- **word embeddings** are vector representations of words that map humanlanguage into a structured geometric space.

EAST TEXAS A&M

L11.4 The Transformer Architecture

CSci 560: Deep Learning w/ Python (Chollet) Ch. 11.4

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

The Transformer Architecture

- Starting 2017 a new model architecture started overtaking recurrent neural networks across most NLP tasks: The Transformer
- Seminal paper "Attention is all you need", Vaswani et al., 2017, 2023
- A simple mechanism called **neural attention** could be used to build powerful sequence models that didn't feature any recurrent layers or convolution layers.
- Neural attention has fast become one of the most influential ides in deep learning.
  - Models should "pay more attention" to some features and "pay less" to others.

EAST TEXAS A&M

Understanding Self-attention

- As you are reading, you may skim some things and attentively read others, depending on your goals or interests.
- What if your deep model could do the same?
- Not all input information seen by a model is equall important to the task at hand.

Similar to some past mechanisms:

- Max pooling in convnets looks at pool of features in spatial region and selects just one feature to keep. All or nothing attention.
- TF-IDF normalization assigns importance scores to tokens based on "information" different tokens are likely to carry. Important bosted, irrelevant get faded out.

EAST TEXAS A&M

Computing Self-attention

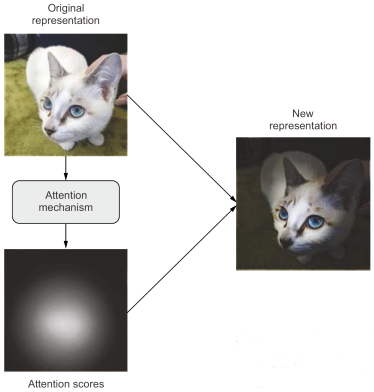


Figure 4: The general concept of "attention" in deep learning: input features get assigned "attention scores," which can be used to inform the next representation of the input. (Chollet, 2021, pg.337)

- Attention can be used for more than just highlighting or erasing certain features.
- Can make features **context-aware**
  - In an embedding space a single word has a fixed position, a fixed relationship with every other word
  - But in reality, the meaning of a word is **context-specific** so position should shift depending on context.
- A smart embedding space would provide different vector representation for a word depending on other words around it.
- self-attention modulates the representation of a token by using the representations of related tokens in the sequence.

EAST TEXAS A&M

Computing Self-attention

1. Compute relevancy scores between the vector “station” and every other word (dot product of the word vectors).
2. Compute sum of all word vectors in sentence weighted by relevance scores. Resulting vector is our new representation that incorporates surrounding context.
3. Repeat for each word in sentence **gives new sequence of vectors encoding the sentence!**

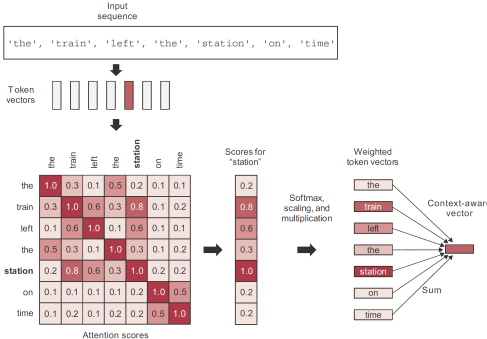


Figure 5: Self-attention scores are computed between "station" and every other word in the sequence, and they are then used to weight a sum of word vectors that becomes the new "station" vector. (Chollet, 2021, pg.338)

Pseudocode Implementation of Self-attention

```
def self_attention(input_sequence):
    """The input_sequence is a sequence of encoded vectors from
    a word embedding like space.  For example, for 600 words encoded
    in a 100d embedding, it would be shape (600, 100)

    Parameters
    ----------
    input_sequence : numpy array shape (max_tokens, dimensions)
        A single sequence, like a sentence for text, but encoded in a word embedding like space. Each
        sample is the vector of embeddings for a single token.

    Returns
    -------
    output : numpy array shape (max_tokens, dimensions)
        Same shape array of token vectors returned, but all of the token vectors has been weighted and
        summed by attention, so each is now a context-aware token vector.
    """
    output = np.zeros(shape=input_sequence.shape)
    # iterate over each individual token from 0..max_tokens-1
    for i, pivot_vector in enumerate(input_sequence):

        # step 1 compute relevancy scores e.g. attention
        # compute the attention score, which is dot product between the token
        # and every other token in this input sequence
        scores = np.zeros(shape=(len(input_sequence),))
        for j, vector in enumerate(input_sequence):
            scores[j] = np.dot(pivot_vector, vector.T)

        # scale by a normalization factor and apply softmax, result is probability
        # distribution that sums up to 1, but still basically importance scores
        scores /= np.sqrt(input_sequence.shape[1])
        scores = softmax(scores)

        # now step 2, compute sum of all word vectors, weighted
        # by relevancy, resulting in new representation of this token
        new_pivot_representation = np.zeros(shape=pivot_vector.shape)
        for j, vector in enumerate(input_sequence):
            new_pivot_representation += vector * scores[j]

        # so on output, the vector representation for token i is the new
        # context-aware representation
        output[i] = new_pivot_representation
    return output
```

EAST TEXAS A&M

Generalized Self-attention: query-key-value model

- A Transformer is a **sequence-to-sequence** model, it was designed to convert one sequence into another.
- The self-attention mechanism performs the following schematically
  `outputs = sum(inputs * pairwise_scores(inputs, inputs))`
- For each token in inputs (A) compute how much it is related to inputs (B)
- Use these scores to weight a sum of tokens from inputs (C)
- There is nothing that requires A, B, C to be the same inputs.
  - call them query-keys-values
    `outputs = sum(values * pairwise_scores(query, keys))`

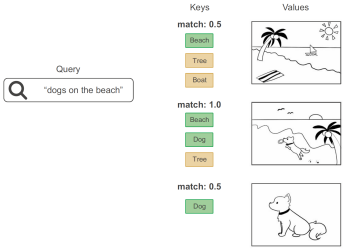


Figure 6: Retrieving images from a database: the "query" is compared to a set of "keys," and the match scores are ued to rank "values". (Chollet, 2021, pg.341)

EAST TEXAS A&M

Multi-head Attention

- Output space of the self-attention layer gets factored into independent subspaces: query, key and value.
- Each vector processed via neural attention, the three outputs are concatenated back together into a single output.
- Each such subspace is called a "head."
  - Having independent heads helps the layer learn different groups of features for each token.

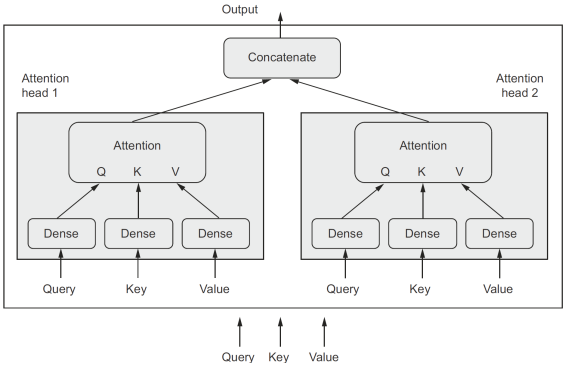


Figure 7: The Keras MultiHeadAttention layer. (Chollet, 2021, pg.344)

Transformer Encoder Architecture

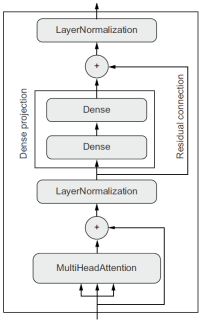


Figure 8: The TransformerEncoder chains a MultiHeadAttention layer with a dense projection and adds normalization as well as residual connections. (Chollet, 2021, pg.344)

Roughly the architecture of Transformer encoder

- Factoring outputs into multiple independent spaces
- Adding residual connections
- Adding normalization layers

Together these form the **Transformer encoder** one of two critical parts that make up the transformer architecture.

EAST TEXAS A&M

Features of NLP Models

- Self-attention is a set-processing mechanism.
- So why say Transformer is a sequence model?
  - Have to add **positional encoding**

	Word order awareness	Context awareness (cross-words interactions)
Bag-of-unigrams	No	No
Bag-of-bigrams	Very limited	No
RNN	Yes	No
Self-attention	No	Yes
Transformer	Yes	Yes

EAST TEXAS A&M

When to use Sequence Models vs. Bag-of-word Models

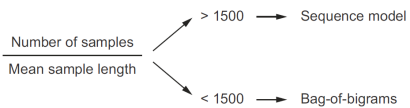


Figure 9: A simple heuristic for selecting a text-classification model: the ratio between the number of training samples and the mean number of words per sample. (Chollet, 2021, pg.349)

- May hear that bag-of-word approach is outdated and should be using sequence models for text processing.
- This is not true, a small stack of `Dense` layers with bigrams was best we got for IMDB sentiment classifier.

The argument why is:

- The input of a sequence model represents a richer and more complex space, and thus it takes more data to map out that space
- Meanwhile, a plain set of terms is a space so simple that you can train a logistic regression on top using just a few hundreds or thousands of samples.
- In addition, the shorter a sample is, the less the model can afford to discard any of the information it contains, in particular word order becomes more important.

EAST TEXAS A&M

Summary of the Transformer Architecture

- **Neural attention** can be used to build powerful sequence models that don't feature recurrent layers.
- Make features **context-specific** by computing attention.
- A smart embedding space provides different vector representations for a word depending on the other words around it.
- `MultiHeadAttention` layer takes 3 inputs, all using a word embedding encoding. It outputs embeddings after factoring in self-attention for the sequences.
- `TransformerEncoder` layer uses multi-head attention as input, and factors outputs into multiple inddpendent spaces, adds residual connections and normalization layers.
- Bag-of-word models with bi-grams are sill good choices when you may not have enough samples or sample length is large.
- Sequence models, including transformers, work best with large data and/or small sample size.

EAST TEXAS A&M

L11.5 Beyond Text Classification: Sequence-to-sequence learning

CSci 560: Deep Learning w/ Python (Chollet) Ch. 11.5

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Sequence-to-sequence models

A sequence-to-sequence model takes a sequence as input, and translates it into a different sequence. Some example applications:

- Machine translation—Convert a paragraph in a source language to its equivalent in a target language.
- Text summarization—Convert a long document to a shorter version that retains the most important information.
- Question answering—Convert an input question into its answer.
- Chatbots—Convert a dialogue prompt into a reply to this prompt, or convert the history of a conversation into the next reply in the conversation.
- Text generation—Convert a text prompt into a paragraph that completes the prompt.

EAST TEXAS A&M

Template of Sequence-to-sequence Learning

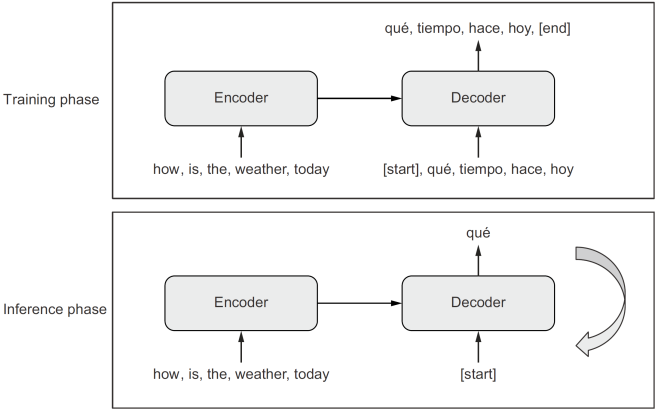


Figure 10: Sequence-to-sequence learning: the source sequence is processed by the encoder and is then sent to the decoder. The decoder looks at the target sequence so far and predicts the target sequence offset by one step in the future. During inference, we generate one target token at a time and feed back into the decoder. (Chollet, 2021, pg.351)

EAST TEXAS A&M

Sequence-to-sequence Training and Inference

General template behind sequence-to-sequence models for training phase:

- An **encoder** model turns the source sequence into an intermediate representation.
- A **decoder** is trained to predict the next token `i` by looking at both previous tokens (0 to `i - 1`) and the encoded source sequence.

During inference, we don't have the target sequence, we're trying to predict it from scratch. We have to generate one token at a time:

1. Obtain the encoded source sequence form the encoder.
2. The decoder starts by looking at the encoded source sequence as well as an initial "seed" token, and predicts the first real token.
3. The predicted sequence so far is fed back into the decoder, which generates the next token and so on, until it generates a stop token.

EAST TEXAS A&M

The Transformer Decoder

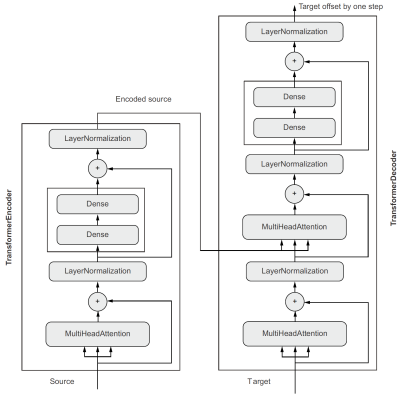


Figure 11: The TransformerDecoder is similar to the TransformerEncoder, except it features an additional attention block where the keys and values are the source sequence encoded by the TransformerEncoder. Together, the encoder and decoder form an end-to-end Transformer. (Chollet, 2021, pg.359)

EAST TEXAS A&M

Summary of Sequnce-to-Sequence Learning

- A sequence-to-sequence model takes a sequence as input and translates it into a different sequence.
- An *encoder* model turns the source sequence into an intermediate representation.
- A *decoder* takes the encoder output and is trained to predict the next token by looking at both all previous tokens, and the encoded source sequence.
- The TransformerDecoder is similar to the TransformerEncoder, but has an additional attention layer that receives the encoded output from an encoder.

EAST TEXAS A&M

Chapter Summary

- There are two kinds of NLP models: bag-of-words models that process sets of words or N-grams without taking into account order, and sequence models that process word order.
  1. A bag-of-words model is made of Dense layers,
  2. while a sequence model could be an RNN, a 1D convnet, or a Transformer.
- Word embeddings are vector spaces where semantic relationships between words are modeled as distance relationships between vectors that represent those words.
- Sequence-to-sequence learning is a framework that can be applied to solve many NLP problems, including machine translation. A sequence-to-sequence model is made of an encoder, which processes a source sequence, and a decoder, which tries to predict future tokens in target sequence by looking at past tokens, using encoder-processed source sequence.
- Neural attention is a way to create context-aware word representations. It's the basis for the Transformer architecture.
- The Transformer architecture yields excellent results on sequence-to-sequence tasks. The first half, the TransformerEncoder, can also be used for text classification or any sort of single-input NLP task.

EAST TEXAS A&M

Bibliography

Chollet, F. (2021). *Deep learning with python* (second). Manning.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need. https://arxiv.org/abs/1706.03762

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

EAST TEXAS A&M